NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

DUARTE JOSÉ LOURENÇO BELO

Bachelor in Computer Science and Engineering

COST AND OPERATIONS OPTIMIZATION IN MULTI-CLOUD ENVIRONMENTS OVER GENAI FRAMEWORKS

MASTER IN COMPUTER SCIENCE AND ENGINEERING

SPECIALIZATION IN SPECIALIZATION NAME

NOVA University Lisbon

Draft: *June 29, 2026*

NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

DEPARTMENT OF
COMPUTER SCIENCE

COST AND OPERATIONS OPTIMIZATION IN MULTI-CLOUD ENVIRONMENTS OVER GENAI FRAMEWORKS

DUARTE JOSÉ LOURENÇO BELO

Bachelor in Computer Science and Engineering

Supervisor: Tomás Hipólito
PhD, Closer Consulting

Co-supervisor: Margarida Mamede
Professor, NOVA School of Science and Technology

MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN SPECIALIZATION NAME

NOVA University Lisbon
Draft: *June 29, 2026*

Abstract

Keywords: Multi-Cloud · Generative AI · Cost Optimisation · Cloud Operations · GenAI Frameworks

Resumo

Palavras-chave: Multi-Cloud · Inteligência Artificial Generativa · Otimização de Custos · Operações em Nuvem · Frameworks GenAI

Disclosure of Delegation to Generative Artificial Intelligence

Duarte José Lourenço Belo declares[1] the use of Generative AI in the research work and writing process of this document. According to the GAIDeT taxonomy [34], the following tasks were delegated to Generative AI tools under full human supervision:

Conceptualization

- ☐ Idea generation
- ☐ Defining the research objective
- ☑ Formulating research questions and hypotheses
- ☐ Feasibility assessment and risk evaluation
- ☐ Preliminary hypothesis testing

Literature Review

- ☑ Literature search and systematization
- ☑ Writing the literature review
- ☐ Analysis of market trends and/or patent environment
- ☐ Evaluation of novelty and identification of gaps

Methodology

- ☐ Research design
- ☐ Development of experimental or research protocols
- ☐ Selection of research methods

Software Development and Automation

- ☑ Code generation
- ☑ Code optimization
- ☐ Process automation
- ☐ Creation of algorithms for data analysis

Data Management

- ☐ Data collection
- ☐ Validation
- ☑ Data cleaning
- ☐ Data curation and organization
- ☐ Data analysis
- ☐ Visualization
- ☐ Reproducibility testing

[1] This declaration was generated using LuaLaTeX [35] and the LaTeX package `aidisclose` [25].

Writing and Editing

- [x] Text generation
- [ ] Proofreading and editing
- [x] Summarizing text
- [ ] Formulation of conclusions
- [ ] Adapting and adjusting emotional tone
- [x] Translation
- [x] Reformatting
- [ ] Press releases and outreach materials

Ethics Review

- [ ] Bias analysis and discrimination assessment
- [ ] Ethical risk analysis
- [ ] Monitoring compliance with ethical standards
- [ ] Data confidentiality monitoring

Supervision

- [ ] Quality assessment
- [x] Trend identification
- [ ] Identification of limitations
- [ ] Recommendations
- [ ] Publication support

Generative AI tools used: ChatGPT-5.5, Claude 4.6 Sonnet, and Perplexity.

Contents

List of Figures

List of Tables

Acronyms

API application programming interface *(pp. 3, 9–11, 17, 18, 25, 26)*
AWS Amazon Web Services *(pp. 16, 17)*

CapEx capital expenditure *(p. 16)*

DPR Dense Passage Retriever *(p. vii, 11)*

FCT NOVA School of Science and Technology *(p. 31)*
FinOps cloud financial operations *(pp. 9, 17, 23)*

GCP Google Cloud Platform *(pp. 16, 17)*
GenAI generative artificial intelligence *(pp. vii, viii, 1–4, 6–8, 13, 15, 16, 21, 22, 24)*
GRU Gated Recurrent Unit *(p. 8)*

IaC Infrastructure as Code *(p. 17)*

LLM large language model *(pp. vii, 1–3, 5, 7–11, 13, 15, 17–28, 30, 32)*
LLMOps large language model operations *(pp. 7, 23)*
LSTM Long Short-Term Memory *(p. 8)*

MCP Model Context Protocol *(pp. vii, 1–3, 5, 7, 8, 13, 14, 26–28, 30, 32)*
ML machine learning *(p. 22)*

OpEx operational expenditure *(p. 16)*

PII personally identifiable information *(p. 15)*
PPO Proximal Policy Optimization *(p. 10)*

RAG retrieval-augmented generation *(pp. vii, 4, 7, 8, 11–13, 15, 18–20, 22, 24–26, 29)*

RAGAS Retrieval Augmented Generation Assessment *(pp. 5–7, 22, 28, 30, 32)*
RAGOps retrieval-augmented generation operations *(pp. 22, 23)*
ReAct reasoning and acting *(pp. vii, 13–15)*
RLHF Reinforcement Learning from Human Feedback *(p. 10)*

SBERT Sentence-BERT *(p. 12)*
SFT supervised fine-tuning *(p. 10)*

1

Introduction

1.1 Context & Motivation

As enterprise adoption of GenAI transitions from exploratory pilots to sustained production use, organisations face a growing set of operational challenges that existing frameworks do not fully address. The rapid proliferation of LLM-based capabilities has exposed critical gaps in how intelligent applications are built, integrated, and governed in production environments: fragmented tool connectivity, limited cost visibility, and insufficient orchestration flexibility [6].

It is precisely in this operational landscape that the present work is situated. The research is grounded in, and developed around, a concrete production-made platform that materialises these requirements in day-to-day production use: Maestro, an enterprise GenAI framework developed at Closer Consulting and deployed in multi-cloud environments. Maestro was conceived to address the operational realities outlined above by providing a structured layer for deploying and managing GenAI workloads across heterogeneous cloud infrastructure. Because it is designed to be instantiated across distinct production environments, the framework must be simultaneously adaptable and robust: it should accommodate diverse model providers, integration patterns, and infrastructure configurations without requiring fundamental redesign at each deployment.

Despite this ambition, three concrete problems currently limit the framework's effectiveness in production. First, tool and service integration remains brittle. In the absence of a standardised connectivity protocol, each new data source or capability requires bespoke integration work, increasing development overhead and reducing portability across deployments [2]. The MCP, introduced by Anthropic in 2024, offers a compelling solution to this problem by defining a universal interface through which LLM hosts can discover and invoke external tools and resources [2]. Integrating MCP into Maestro would enable a plug-and-play connectivity model that scales across production environments without duplicating integration effort.

Second, cost observability is severely constrained. In the current deployment context, LLM infrastructure costs are consolidated under shared cloud resources, with no segmentation by agent, workflow, or request type. This architectural reality makes it impossible to

attribute expenditure to specific system components, which in turn prevents principled optimisation decisions. Without granular cost telemetry in terms of token consumption at the level of individual invocations, teams operating Maestro in production cannot identify inefficient patterns, enforce budget boundaries, or demonstrate the cost-effectiveness of GenAI capabilities to stakeholders [41]. The absence of cost observability is not merely an inconvenience; it is a structural barrier to responsible scaling.

Third, multi-agent orchestration within Maestro lacks the composability required to support complex, interdependent workflows. As GenAI applications grow in scope, they increasingly require the coordination of specialised agents operating in parallel or in structured sequences. Current orchestration approaches do not provide sufficient primitives for expressing these dependencies reliably, which constrains the range of applications that can be built on top of the framework [40].

This dissertation addresses these three problems by proposing a unified and operationally grounded approach. Concretely, the work extends Maestro along three architectural axes: (i) the design and implementation of an MCP client layer that replaces the current set of bespoke connectors with a standardised, plug-and-play integration model; (ii) the instrumentation of the platform with a unified LLM and infrastructure cost telemetry pipeline, enabling per-invocation token and cost accounting; and (iii) the refactoring of multi-agent workflows around a composable agent-graph orchestration runtime. Together, these extensions are accompanied by a methodology for evaluating production-oriented GenAI infrastructure under operationally realistic conditions.

1.2 Institutional Context

The research presented in this dissertation emerges from a partnership between the Department of Computer Science at NOVA School of Science and Technology (NOVA FCT) and Closer Consulting, which serves as the main industrial collaborator. As the author, I have assumed a dual role - both as an academic researcher and as an active practitioner within Closer Consulting - enabling unique integration of theoretical inquiry and practical implementation.

Closer Consulting is responsible for the design, development, and maintenance of the Maestro GenAI framework, which represents both the technical foundation and the core object of study of this work, functioning simultaneously as the starting point, the engineering substrate on which contributions are built, and the environment in which they are evaluated. Throughout the research process, I have been granted privileged access to Maestro's production deployments, operational cost telemetry, and multi-tenant enterprise configurations, facilitating a level of analysis and validation grounded in production conditions [10].

This collaboration shapes the research agenda directly: architectural decisions, implementation choices, and evaluation metrics are grounded in enterprise constraints leaving the research grounded in production conditions.

1.3 Problem Statement

Despite the growing maturity of enterprise GenAI development, frameworks designed for production use continue to face a set of structural limitations that constrain their scalability, operational efficiency, and portability across deployment environments. This section identifies the three core challenges that motivate the present work, all of which are grounded in the operational reality of Maestro, a framework for enterprise GenAI applications in multi-cloud environments developed at Closer Consulting [10]. Each challenge is mapped explicitly to one or more of the four research objectives stated in Section 1.4: Challenge 1 motivates O1, Challenge 2 motivates jointly O2 and O3, and Challenge 3 motivates O4.

Challenge 1 - Fragmentation of connectivity (motivates O1). The first challenge concerns the standardisation of tool and service integration. Modern enterprise GenAI systems are required not only to consume data from a wide variety of sources - databases, cloud services, analytics platforms, and enterprise application programming interfaces (APIs) - but also to integrate with the capabilities exposed by external technical tools and APIs across heterogeneous infrastructure. In Maestro's current architecture, every connection of this kind is implemented as a bespoke connector, with no shared interface contract governing how tools and resources are discovered or invoked by the underlying LLM [10]. In the absence of such a shared protocol, integration effort is repeated for every new service, the resulting code becomes tightly coupled to the specific provider being consumed, and the system as a whole is difficult to port across deployments without substantial rework - undermining Maestro's multi-cloud design goals. The MCP, introduced by Anthropic in 2024, proposes a universal, open protocol that standardises how LLM hosts discover and invoke external tools and resources [2, 27]. However, the integration of MCP into production-grade GenAI frameworks - including the definition of reusable client implementations across cloud platforms - remains an open engineering challenge with limited empirical treatment in the literature [36, 1].

Challenge 2 - Absence of cost observability (motivates O2 and O3). The second challenge concerns cost observability and operational efficiency. In the current deployment context of Maestro, LLM costs are consolidated under a shared cloud bill, with no detail per agent, workflow, or request type. The framework, as deployed today, performs no form of cost or token-consumption analysis: there is no mechanism to determine what costs most, no means of attributing expenditure to specific system components, and consequently no way to justify the investment in GenAI capabilities to stakeholders [41]. Beyond this lack of segmentation, several technical dimensions of cost observability are also absent and are nonetheless essential for the projects built on top of the framework, namely:

1. per-invocation accounting of input, output, and total token counts, together with the resulting monetary cost per model;

2. attribution of those costs to the originating tenant, agent, workflow, prompt template, and RAG pipeline stage;

3. observability of cache behaviour, including semantic-cache hit and miss rates and their effect on token consumption;

4. latency, throughput, and error-rate signals correlated with cost, so that performance and expenditure can be reasoned about jointly; and

5. time-series aggregation that supports budget enforcement, anomaly detection, and longitudinal cost trend analysis.

This absence is not merely a financial inconvenience: it is a structural barrier to making data-driven decisions about which optimisation techniques to apply. Without granular telemetry at the level of individual invocations it is impossible to characterise the production workload - query repetition rate, dominant prompt patterns, average completion length per agent, distribution of model usage - and therefore impossible to select cost-reduction techniques on principled grounds. For this reason, instrumentation (O2) is treated as a hard prerequisite for any cost-reduction work (O3) rather than as a parallel concern. The literature describes a broader set of techniques relevant to O3 - including prompt compression [17, 29], semantic caching [5], dynamic model routing [9], and RAG pipeline optimisation [14] - none of which have been evaluated or implemented within Maestro. A subset of these techniques is later examined in this dissertation; the comparison with prior work is deferred to Chapter **??**, where the approach adopted here is positioned both qualitatively and, where measurements allow, quantitatively against the techniques discussed above. The combined lack of measurement infrastructure and applied optimisation represents a compounded barrier to responsible scaling in production [3].

Challenge 3 - Orchestration without composability (motivates O4). The third challenge concerns orchestration composability for multi-agent workflows. As GenAI applications evolve from single-turn question-answering systems towards complex, multi-step reasoning pipelines, the need for robust multi-agent orchestration primitives becomes increasingly critical [40]. These can vary from simple sequence execution to complex conditional branching and parallel processing. Maestro's agents, however, have been built case by case - for sentiment analysis, session summarisation, relevance checking, and similar tasks - without a shared structural model [10]. Adding new functionality therefore requires modifying existing code rather than composing new behaviour from reusable units, which limits the range of expressible workflows, prevents component reuse, and offers no standard mechanism for inter-agent communication, controlled parallelism, or delegation of subtasks across agents. Equally important, the current implementation provides no execution tracing of what each agent does at runtime: there is no per-step record of inputs, outputs, intermediate decisions, or tool invocations, which makes debugging and quality assurance disproportionately difficult in production and prevents any systematic analysis of where

workflows succeed or fail. As a consequence, the framework cannot readily support the class of complex, interdependent multi-agent applications that are increasingly demanded in enterprise settings [40].

1.4 Objectives

Four research objectives are proposed to address the structural limitations identified in Section 1.3. Each is stated as a concrete, measurable outcome with explicit success criteria and a designated primary toolset. They are not guaranteed to be fully achieved: all four are pursued within the time constraints of the dissertation, and execution follows a fixed order of dependency and priority.

O1 is self-contained and is implemented first. O2 is implemented second and is a hard prerequisite for everything that follows, as it establishes the cost and quality baseline without which O3 and O4 cannot be evaluated. Once O2 is complete, O3 is the primary focus - it addresses the cost-reduction goals central to this dissertation. O4 is only pursued if time remains after O3 is substantially complete, as it represents an independent architectural improvement rather than a dependency of O3.

Although Maestro is designed for multi-cloud deployment, the implementation work undertaken in this dissertation is carried out against a single concrete cloud environment, namely Microsoft Azure, which is the production environment in which the framework is currently operated. Objectives and contributions are therefore framed in terms of the artefacts produced on Azure and the architectural patterns that emerge from them, rather than claims of equivalence across cloud providers. Where validation is still in progress at the time of writing, safegarded language is used to indicate that the stated outcome remains a hypothesis pending empirical confirmation.

O1 - MCP client integration. The first objective is to introduce, end-to-end, the MCP into Maestro as a first-class connectivity primitive[2, 27]. Maestro will consume external MCP servers - such as MongoDB, Microsoft Learn, and Fabric MCP - without bespoke per-provider integration code, relying solely on protocol-level discovery and invocation.

O2 - Unified cost telemetry pipeline. The second objective is to instrument Maestro so that, for every LLM invocation, the platform automatically records cost, execution trace, and output quality, thereby producing the first quantitative baseline for the platform. To the best of the author's knowledge, no equivalent cost-observability pipeline currently exists for Maestro, and this baseline is treated as a hard prerequisite for the cost-reduction work undertaken in O3. Langfuse will serve as the LLM observability layer, while Azure Monitor will aggregate infrastructure-level signals via OpenTelemetry, providing a unified operational view across both LLM and compute dimensions on the Azure-hosted deployment of Maestro. Output quality is sampled using the Retrieval Augmented Generation Assessment (RAGAS) evaluation framework [12]. The objective will be considered fulfilled when (i) one hundred per cent of the LLM invocation paths identified in the Maestro code base are instrumented via Langfuse; (ii) each invocation exports its cost per request, its

execution trace, and, in sampling mode, its RAGAS quality score; (iii) the costs reported by Langfuse exhibit less than five per cent deviation from the actual Azure billing invoice for the same period, validating the reliability of the baseline; and (iv) the production baseline is available before any O3 intervention begins.

O3 - Cost reduction via tokenomics optimisation. The third objective is to implement and empirically compare at least two complementary cost-reduction techniques, selected on the basis of the patterns identified in the O2 baseline rather than chosen a priori. The selection is therefore data-driven: a workload exhibiting a high query repetition rate motivates semantic caching, whereas a workload dominated by simple queries motivates cascade routing towards smaller models. The candidate techniques are prompt compression via LLMLingua [17], semantic caching via MongoDB Atlas Semantic Cache or Azure Managed Redis (conceptually grounded in GPTCache [5]), and cascade routing via LangChain model routing [8] (inspired by the FrugalGPT cascade strategy [9]). To structure this analysis, a *three-layer tokenomics framework* - presented in detail in Section 2.6.6 - will aim to decompose the cost of a GenAI invocation into the input layer (the tokens sent to the model), the model layer (the cost characteristics of the model that processes the request), and the output layer (the tokens returned to the caller); each candidate technique targets a different layer of this framework. The objective will be considered fulfilled when (i) the selected techniques achieve a significant reduction in cost-per-query relative to the O2 baseline; (ii) quality equivalence relative to the baseline is demonstrated on RAGAS faithfulness and answer relevance scores via Two One-Sided Tests (TOST) [19] at $\alpha = 0.05$ with an equivalence margin of ± 0.05; and (iii) the evaluation is conducted on real production queries, partitioned into a control group and an experimental group. TOST is adopted in preference to a standard two-sample t-test because the research question is one of equivalence - showing that the experimental and control distributions are sufficiently close to be considered indistinguishable - whereas a non-significant t-test only fails to detect a difference and cannot, on its own, support a positive claim of quality preservation [20].

O4 - Composable agent graph orchestration. The fourth objective is to refactor Maestro's agents as reusable, testable LangGraph [8] graphs, sharing the same observability stack as O2. LangGraph is selected for three reasons that are architecturally significant. First, Maestro already uses LangChain [8] in production; LangGraph is LangChain's native graph orchestration runtime, making it an extension of the existing stack rather than a replacement, and thereby minimising migration risk. Second, Azure AI Foundry natively hosts LangGraph agents, preserving alignment with the existing Azure infrastructure. Third, Langfuse integrates natively with LangGraph, so that O2 and O4 share a single observability stack - a deliberate architectural decision that eliminates the cost and complexity of maintaining parallel observability pipelines, rather than a limitation imposed by tooling. The objective will be considered fulfilled when (i) at least two production workflows have been refactored as LangGraph graphs with structurally distinct topologies - one sequential and one with conditional branching or parallelism - demonstrating that the LangGraph pattern is expressive enough for diverse real-world cases; (ii) trace coverage

of at least eighty per cent of execution steps is achieved via Langfuse; (iii) a reduction of at least thirty per cent in lines of code is achieved relative to the original hand-crafted implementations; and (iv) the refactored agents are accompanied by unit tests, where the original implementations had none.

1.5 Expected Contributions

Pursuing the four research objectives stated in Section 1.4 is expected to produce the following five original contributions:

- **MCP client integration layer for Maestro.** An MCP client module enabling Maestro to consume external MCP-compliant services without bespoke connectors [2, 27].
- **Unified LLM observability pipeline.** The first integrated observability pipeline for Maestro, covering cost per invocation, multi-step agent execution traces, and output quality scores, implemented via Langfuse and Azure Monitor over OpenTelemetry and validated against production billing data.
- **Empirical benchmarks of cost-reduction techniques.** A statistically rigorous evaluation (TOST, $\alpha = 0.05$) of at least two tokenomics techniques selected on the basis of production usage patterns, conducted on real production queries - providing ecologically valid efficacy estimates that controlled laboratory settings cannot replicate.
- **Quantitative large language model operations (LLMOps) baseline.** A production-derived baseline of token cost, latency, and quality metrics (RAGAS) for a real multi-cloud, multi-tenant RAG system, acting both as a hard prerequisite for O3 and as a reproducible evaluation reference for future comparative studies [12].
- **Reference architecture for composable, observable, and cost-aware GenAI frameworks.** A documented design pattern for enterprise GenAI orchestration in multi-cloud environments, comprising composable LangGraph agent graphs, a shared Langfuse and OpenTelemetry observability stack, and data-driven cost optimisation - applicable beyond the Maestro platform.

1.6 Structure

The remaining of this document is organised as follows: Chapter 2 reviews the relevant background on GenAI and LLM orchestration frameworks, MCP, and cost optimisation techniques in a production context. Chapter **??** describes the Maestro framework, the problems that motivate the research and the methodology applied to address them. Chapter **??** presents the research objectives and the expected contributions. Chapter **??** presents the evaluation of the research contributions. And finally, chapter 4 presents the work plan and the expected contributions.

2

Background

This chapter provides the background necessary to situate the research contributions. Section 2.1 introduces the foundations of neural language representation. Section 2.2 characterises large language models in production. Section 2.3 covers RAG, followed by Section 2.4, which addresses LLM orchestration frameworks, the MCP, and the Maestro enterprise GenAI framework. Sections 2.5 and 2.6 address multi-cloud infrastructure and cost optimization strategies, respectively. Section 2.7 covers operational governance of GenAI systems.

2.1 Foundations of Neural Language Representation

Two landmark contributions established the foundations of contemporary LLMs and retrieval-augmented architectures: dense word representations learned via neural networks, and the Transformer architecture based solely on attention.

2.1.1 Distributed Word Representations

Dense word representations, established by Mikolov et al. [26], demonstrated that semantic similarity can be captured as geometric proximity in a learned continuous vector space - a principle that directly underlies the embedding models and vector databases central to modern RAG pipelines. This representational substrate informs the quality and cost of the embedding stage in the Maestro framework's RAG pipeline, where the choice of embedding model has direct implications for retrieval precision and token expenditure.

2.1.2 The Transformer Architecture

Sequence modelling with recurrent architectures such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) suffered from sequential processing and vanishing gradients over long contexts. Vaswani et al. [39] proposed the Transformer, which dispenses with recurrence and relies solely on attention to model dependencies across all positions simultaneously. Its core operation, scaled dot-product attention, computes outputs as a weighted sum of value vectors:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

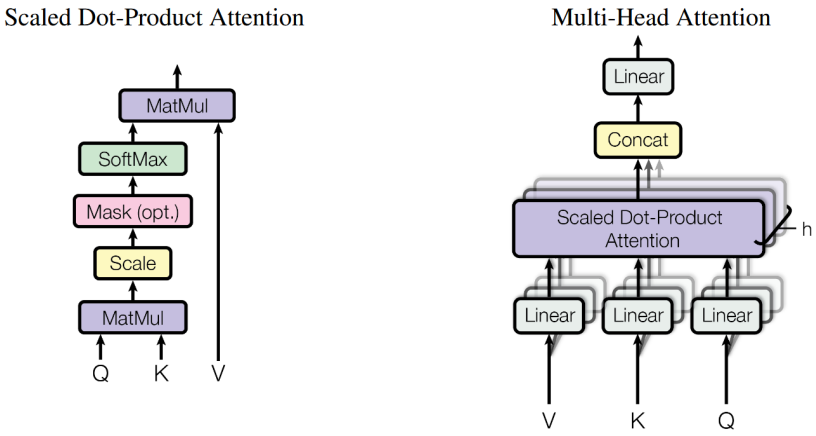

Figure 2.1: Scaled Dot-Product Attention (left) and Multi-Head Attention (right). Adapted from [39].

Multi-Head Attention runs several such operations in parallel, allowing the model to attend to different types of relationships simultaneously. Positional encodings compensate for the architecture's permutation invariance. The resulting model achieved 28.4 BLEU on the WMT 2014 English-to-German benchmark, surpassing all prior approaches at a fraction of the training compute [39].

The Transformer's parallelism unlocked a new scaling regime, yielding encoder-only models such as BERT [11] and decoder-only models such as the GPT series [6]. Crucially, attention scales quadratically with sequence length, $O(n^2)$, making context length a primary cost driver in token-priced LLM APIs. This property is foundational to the cloud financial operations (FinOps) strategies explored in this dissertation - prompt compression, semantic caching, and dynamic model selection - each of which seeks to reduce the effective n processed per inference [39].

2.2 Large Language Models in Production

Building on distributed representations and the Transformer, the field consolidated around the LLM: large decoder-only Transformer networks, pre-trained on internet-scale corpora, that acquire general-purpose competence redirectable to downstream tasks with minimal supervision. This section characterizes LLMs as a class, traces their evolution from pre-training to instruction-following alignment, and examines the deployment spectrum relevant to enterprise cost optimization.

2.2.1 Large Language Models: An Overview

A LLM is a neural language model of sufficient parameter count and training-data scale that general linguistic and reasoning capabilities emerge from the pre-training objective alone [6]. Pre-training proceeds via self-supervised next-token prediction over corpora of hundreds of billions to trillions of tokens, yielding weights that encode syntax, semantics, world knowledge, and rudimentary reasoning, elicited through natural-language prompting.

A defining empirical finding is the existence of *scaling laws*: power-law relationships between model size, training compute, dataset volume, and downstream performance [6]. Because performance is predictable and improvable by increasing any of these axes, compute investment translates into measurable capability gains. This has driven successive scaling of commercial models and made token consumption a dominant cost variable, directly motivating the cost optimization strategies central to this work.

2.2.2 From Pre-training to Instruction Tuning

Early LLMs exhibited systematic misalignment between model behavior and user intent: a model trained only to predict the next token continues text plausibly, but not necessarily helpfully or harmlessly. Ouyang et al. [28] addressed this gap with a three-stage Reinforcement Learning from Human Feedback (RLHF) pipeline: (i) supervised fine-tuning (SFT) on high-quality human demonstrations, (ii) training a reward model to predict human preferences over output pairs, and (iii) optimizing the fine-tuned model against this reward using Proximal Policy Optimization (PPO). The resulting InstructGPT models, with as few as 1.3 billion parameters, were consistently preferred over the unaligned 175-billion parameter GPT-3 baseline, confirming that alignment quality is not strictly a function of scale [28]. This establishes that a smaller, well-aligned model can match or exceed a larger unaligned one - a relationship that directly informs the dynamic model selection strategies explored in this dissertation.

2.2.3 Open-Weight Models and the Deployment Spectrum

The commercial LLM landscape is stratified between proprietary API-accessed models and self-hostable open-weight alternatives. The former, exemplified by OpenAI's GPT-4 family and Anthropic's Claude series, expose inference through token-priced cloud APIs; the latter is most prominently represented by Meta AI's Llama series. Touvron et al. [38] introduced Llama 2, a family of 7 to 70 billion parameter models pre-trained on two trillion tokens and released for commercial use, with RLHF-aligned variants (Llama 2-Chat) achieving performance competitive with closed-source alternatives while remaining locally deployable [38].

Capable open-weight models introduce a consequential degree of freedom into enterprise architectures. Organizations may route inference dynamically across a portfolio of models differentiated by capability tier, latency, and per-token cost - serving high-volume,

lower-complexity queries with smaller on-premise models while reserving frontier API models for peak-reasoning tasks. This *LLM cascading* paradigm is a central cost optimization mechanism evaluated in this dissertation in the context of Maestro's multi-cloud deployment [38, 28].

2.3 Retrieval-Augmented Generation

LLM parametric knowledge is frozen at training time, cannot be transparently audited, and tends to produce hallucinations when queries exceed the training distribution [23]. RAG addresses these limitations by coupling a generative model with a dynamic, non-parametric memory that can be queried, updated, and inspected at inference time.

2.3.1 Foundational Architecture

The canonical RAG formulation was introduced by Lewis et al. [23], who proposed a fine-tuning recipe for generative models endowed with external non-parametric memory. The architecture comprises two components: (i) a *retriever* $p_\eta(z \mid x)$, implemented as a DPR [18] that maps queries and documents into a shared dense vector space, and (ii) a *generator* $p_\theta(y_i \mid x, z, y_{1:i-1})$, implemented as BART-large [22], which conditions its output on the concatenation of the query x and retrieved passages z.

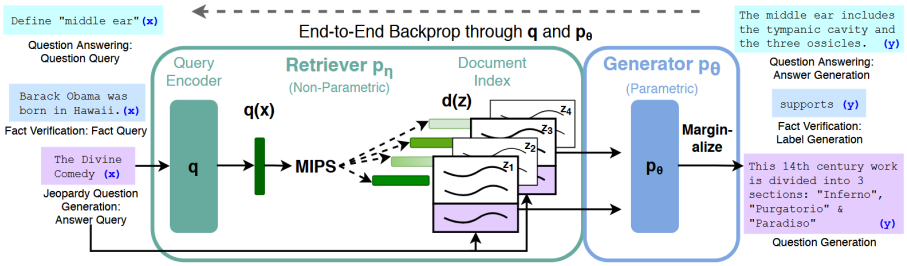

Figure 2.2: End-to-end RAG architecture combining a DPR retriever with a BART-large generator. Adapted from [23].

The retriever and generator are trained end-to-end by minimizing the negative marginal log-likelihood without direct supervision on which documents to retrieve. The document encoder and its FAISS index are frozen during fine-tuning, substantially reducing cost. RAG established state-of-the-art results on open-domain question answering benchmarks, and an index-hotswapping experiment demonstrated that replacing the document index updates the model's world knowledge without retraining - a property of practical significance for production deployments [23].

2.3.2 Architectural Evolution: Naive, Advanced, and Modular RAG

Gao et al. [14] systematize the evolution of RAG methods into a three-tier taxonomy.

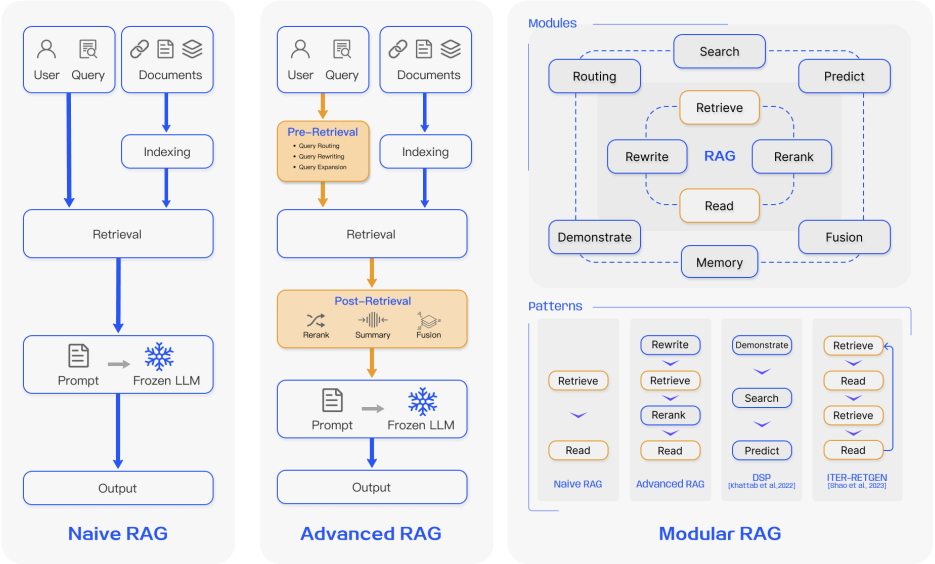

Figure 2.3: Three-tier RAG taxonomy: Naive, Advanced, and Modular RAG. Adapted from [14].

Naive RAG. The earliest implementations follow a minimal *Retrieve-then-Read* pipeline: documents are chunked, encoded, and stored in a vector database; the top-K chunks most similar to the query are retrieved via cosine similarity and concatenated with the query to form a prompt [14]. While effective in simple scenarios, Naive RAG suffers from low retrieval precision, context-window saturation, and hallucinations when the generator defaults to parametric knowledge amid noisy context.

Advanced RAG. Advanced RAG introduces targeted pre- and post-retrieval interventions to address these limitations [14]. Pre-retrieval strategies include query rewriting and sub-question expansion; post-retrieval strategies include cross-encoder reranking and context compression. These modifications retain the linear Retrieve-then-Read structure while substantially improving generation quality.

Modular RAG. Modular RAG decomposes the system into composable functional modules - *Search*, *Memory*, *Routing*, *Predict*, and *Task Adapter* - supporting non-sequential, iterative, and self-reflective retrieval patterns that static pipelines cannot accommodate [14].

2.3.3 Dense Semantic Retrieval and LlamaIndex

Efficient semantic retrieval requires fixed-size sentence embeddings amenable to fast similarity search. Reimers and Gurevych [32] addressed this with Sentence-BERT (SBERT), which modifies BERT using Siamese network structures and mean-pooling to produce embeddings comparable via cosine similarity, reducing pairwise sentence comparison from approximately 65 hours to under 5 seconds for a 10,000-sentence corpus [32].

LlamaIndex [24] operationalises the full RAG design space through a modular architecture separating ingestion, indexing, retrieval, and synthesis. Its `VectorStoreIndex` retrieves nodes via cosine similarity and exposes advanced retrieval patterns including semantic routing, cross-encoder reranking, and iterative retrieval agents. In this work, LlamaIndex serves as the RAG runtime within the Maestro framework, with the choice of embedding model determining both retrieval quality and per-query token cost.

2.4 Frameworks, MCP and Maestro

The preceding sections describe the *components* of a modern GenAI system; this section addresses how they are composed, governed, and operated at production scale.

2.4.1 LLM Orchestration Frameworks

LangChain [8] addressed the challenge of composing LLMs with external tools, memory, and retrieval through three primitives: a *chain* (a composable sequence of calls to an LLM, retriever, or tool), a *tool* (any callable capability described in natural language and typed schema), and *memory* modules for stateful multi-turn interactions.

The theoretical foundation for this model-centric loop was established by Yao et al. [43] with the ReAct paradigm (*Reason* + *Act*), which interleaves chain-of-thought reasoning with tool use in a single iterative cycle: the model emits a *Thought*, selects an *Action*, receives an *Observation*, and repeats until a terminal condition is met. On multi-step knowledge-intensive tasks - HotpotQA, FEVER, ALFWorld, and WebShop - ReAct outperformed both chain-of-thought and action-only baselines, showing that tight coupling of reasoning and acting is essential for robust agent behavior [43].

2.4.2 Model Context Protocol

A persistent weakness of this orchestration paradigm is the absence of a standard interface between agents and the systems they invoke. Each integration is bespoke, creating an $N \times M$ problem - N agent frameworks times M external services - generating duplicated adapter code and fragile dependency chains [2].

The MCP, introduced by Anthropic in 2024 [2, 27], addresses this with an open, transport-agnostic protocol standardizing how LLM hosts expose and consume external capabilities. It defines three roles: an *MCP Host* (the application embedding the LLM), an *MCP Client* (managing connections to servers), and an *MCP Server* (a lightweight process exposing capabilities over JSON-RPC 2.0 via standard I/O or HTTP with Server-Sent Events). Capabilities are organized into *Tools* (invocable functions), *Resources* (read-only data artifacts), and *Prompts* (parameterized templates) [2, 27].

Unlike vendor-specific function calling, MCP is stateful and provider-agnostic, supporting dynamic capability discovery: a client can query any server for its current tool and resource catalog without prior knowledge of its implementation. This reduces the

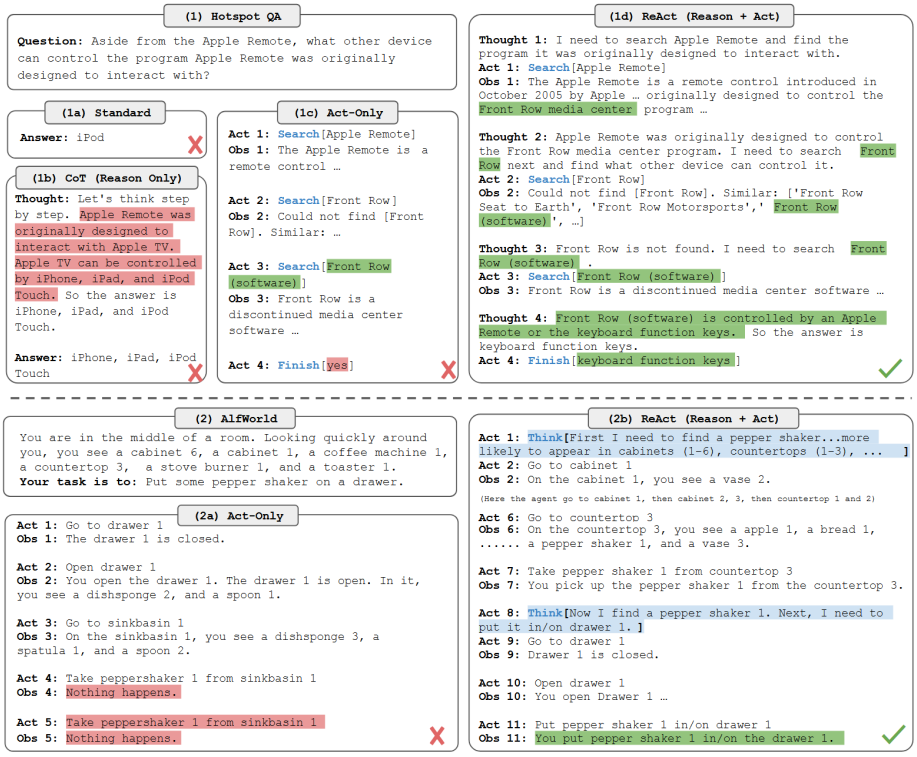

Figure 2.4: The ReAct reasoning loop interleaving *Thought*, *Action*, and *Observation* steps. Adapted from [43].

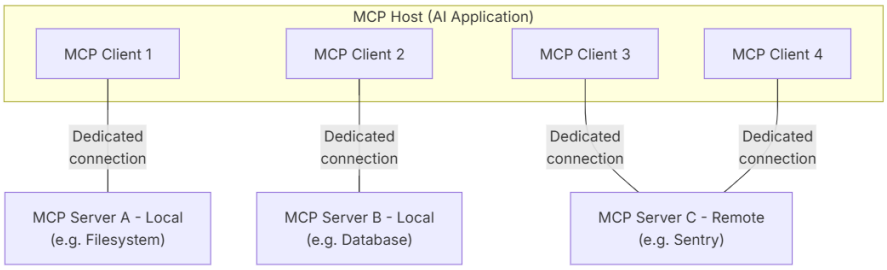

Figure 2.5: MCP architecture showing the Host, Client, and Server roles connected via JSON-RPC 2.0. Adapted from [27].

integration surface from $N \times M$ to $N + M$, and is directly relevant to multi-cloud enterprise deployments where heterogeneous data services must be exposed through a single protocol layer.

2.4.3 Enterprise GenAI Frameworks: Maestro in Context

Enterprise production environments impose requirements that research prototypes do not address: identity-based access control, secret management, encryption, token-level cost observability, multi-tenancy isolation, cross-provider portability, and reliability under variable load. Meeting these needs requires a governed platform in which each component is independently deployable, swappable without contract changes, and instrumented for monitoring and audit compliance.

The Maestro GenAI Framework, developed by Closer Consulting, is a practitioner response to this gap [10]. Validated across production deployments in the energy, healthcare, financial, and pharmaceutical sectors, Maestro is a multi-repository, container-native microservices platform that decomposes the RAG pipeline into independently governed services. The *LLM microservice* routes generation requests to Azure OpenAI, OpenAI, Azure AI Services, and Google Gemini behind a stable contract. The *orchestrator* implements a ReAct-derived RAG loop - history retrieval, hybrid vector search, relevance filtering, prompt assembly, and LLM invocation - as a FastAPI service backed by LangChain. The *vector database service* handles ingestion, chunking, embedding, and hybrid kNN+BM25 retrieval over MongoDB Atlas. Additional microservices provide provider-agnostic persistence (Azure Cosmos DB, MongoDB), personally identifiable information (PII) detection via Microsoft Presidio, and frontend surfaces with Microsoft Entra ID authentication. Infrastructure is provisioned declaratively via OpenTofu; current production deployments operate on Microsoft Azure with observability through OpenTelemetry. In production, the framework has demonstrated measurable impact: one deployment serving over 600 daily active users with 4,000 daily interactions across five LLM models, and another achieving 83% question-answering accuracy over 2,907 indexed legal documents [10].

Despite this maturity, a systematic analysis reveals structural limitations along three dimensions: the absence of a standardized tool-exposure protocol, with each microservice reachable only through bespoke REST endpoints and no dynamic capability discovery; an incomplete cost-observability layer, with no semantic caching or routing to reduce redundant invocations; and an orchestration architecture relying on hand-crafted orchestrators rather than a composable agent graph. These three dimensions constitute the research frontier addressed by the engineering contributions developed in this dissertation.

2.5 Multi-Cloud Computing and Infrastructure as Code

The components surveyed above must be provisioned, networked, secured, and operated on physical or virtual compute. Choices at this layer directly influence cost, resilience, and

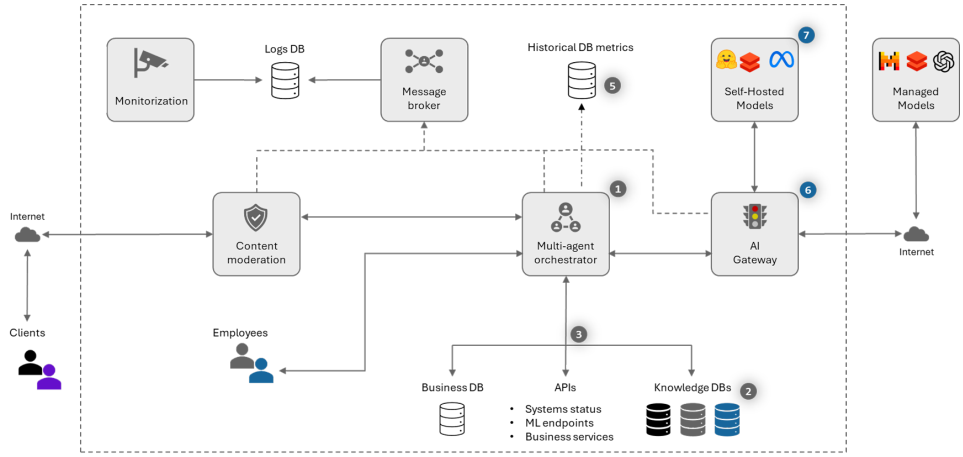

Figure 2.6: Functional architecture of the Maestro GenAI Framework, illustrating the *multi-agent orchestrator*, *AI Gateway*, *content moderation*, self-hosted and managed model endpoints, and observability layer. Adapted from [10].

vendor lock-in - concerns that Maestro treats as first-class design constraints.

2.5.1 Foundations of Cloud Computing

Armbrust et al. [4] identified three properties distinguishing cloud computing from prior paradigms: the *illusion of infinite resources* on demand, the *elimination of upfront capital commitments*, and the ability to *pay for resources short-term* and release them when no longer needed. These enable a shift from capital expenditure (CapEx) to operational expenditure (OpEx), and are particularly valuable for AI-intensive workloads with bursty, hard-to-predict inference load. The same work identified *data lock-in* and *service availability concentration* as persistent obstacles, advocating distribution across independent providers as the principal mitigation - the foundational rationale for multi-cloud architectures and for Maestro's provisioning philosophy across Azure, Amazon Web Services (AWS), and Google Cloud Platform (GCP) [4, 10].

2.5.2 Interoperability and Portability in Multi-Cloud Environments

Petcu [30] formalized multi-cloud engineering challenges by distinguishing *portability* (the capacity to transfer a workload between cloud environments with bounded effort) from *interoperability* (the capacity for components on different clouds to communicate at runtime), identifying four sources of lock-in: proprietary APIs, divergent data formats, incompatible service-level agreements, and asymmetric pricing models - each requiring a distinct mitigation through abstraction layers, protocol normalization, contractual harmonization, and unified billing instrumentation.

Petcu [30]'s taxonomy classifies multi-cloud strategies into three archetypes: *redundant* (one provider active, others standby), *complementary* (distinct capabilities assigned to distinct providers), and *federated* (unified abstraction transparent to the application). Maestro is designed to operate in complementary mode: AWS Bedrock, Azure OpenAI, and Google

Gemini can each be exposed through a uniform `llm-microservice` interface. This stable contract instantiates precisely the abstraction-layer pattern advocated for resolving API heterogeneity [30, 10].

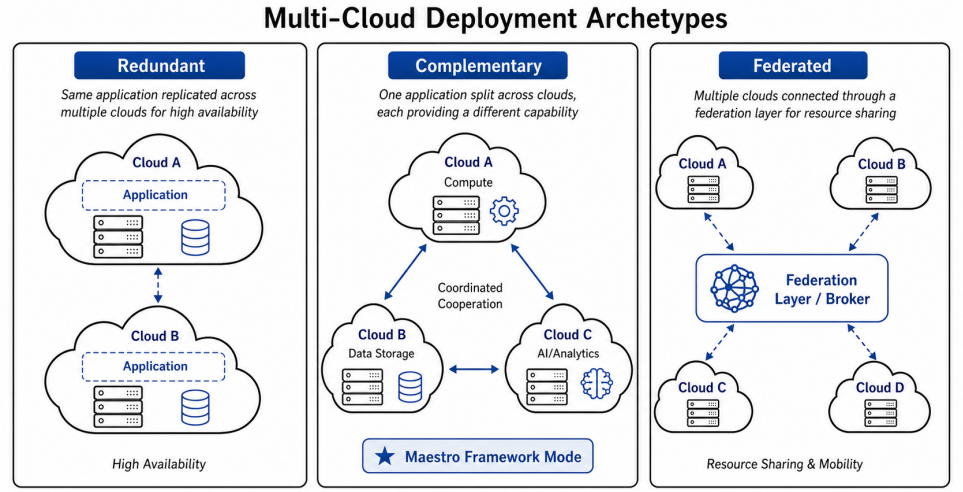

Figure 2.7: Multi-cloud deployment archetypes - redundant, complementary, and federated strategies - as classified by Petcu [30]. The Maestro framework operates in complementary mode.

2.5.3 Infrastructure as Code: Research Landscape and Practice

Infrastructure as Code (IaC) is the practice of expressing computing infrastructure configuration as version-controlled, machine-readable source files interpreted by a provisioning engine, rather than manual operator procedures [31]. The declarative approach - where practitioners specify the desired end state and the engine computes convergence - is the dominant paradigm. Rahman et al. [31] systematically mapped the IaC research landscape, identifying reproducibility, empirical quality assessment, and automated testing as its defining concerns. In the Maestro framework, infrastructure is provisioned via OpenTofu - the Linux Foundation-governed fork of HashiCorp Terraform - with declarative configurations for Azure, AWS, and GCP [10]. Reproducible, version-controlled infrastructure enables consistent cost attribution across environments: every resource is declared in commit history, allowing cost anomalies to be traced to specific changes and supporting the FinOps disciplines of budget forecasting and spend attribution central to this work.

2.6 Cost Optimization

The economic profile of LLM systems is dominated by inference costs that scale with tokens processed, model capability tier, and invocation frequency. In multi-cloud deployments these costs accumulate across providers and workloads, making cost optimization a first-class architectural concern [4, 10]. The unifying lens is *tokenomics*: minimising token

consumption relative to the semantic value delivered per inference call. Because the cost-accuracy trade-off can be shaped by architectural choice - as FrugalGPT demonstrates by achieving GPT-4-level accuracy at a fraction of its cost via adaptive routing [9] - tokenomics is a systems-design discipline. Market forces alone are insufficient: token prices have fallen substantially [3], yet total inference spend keeps growing as prompts and usage expand. Each technique surveyed below is a composable lever over a shared cost function, operating at one of three layers: the *model layer* (cascade routing), the *retrieval layer* (caching and reranking), and the *representation layer* (prompt compression, format, and output style).

2.6.1 Semantic Caching

Many production queries are not unique: users repeat, rephrase, or follow up on earlier requests. Traditional exact-key caches cannot exploit semantic similarity, so even minor paraphrases miss. GPTCache addresses this with an open-source semantic cache that retrieves responses via embedding-based similarity search [5]. Incoming queries are encoded into embeddings and a nearest-neighbour search determines whether a semantically similar query was already answered; on a cache hit the stored response is returned, and only on a miss is the request forwarded to the LLM. Cache hits increase response speed by 2–10× while insulating the application from provider-side latency spikes [5]. Because an embedding lookup costs negligibly compared to a full LLM call, even a modest hit rate yields tangible API savings - particularly for enterprise assistants, FAQ bots, and RAG systems where queries repeat strongly across tenants and time.

2.6.2 LLM Cascades

Where semantic caching reduces redundant calls, LLM cascades target *which* model is invoked when a call is necessary. FrugalGPT studies how to route queries across a portfolio of models to minimize cost subject to accuracy constraints, on the premise that not all requests need the most capable model [9]. Routing strategies range from rule-based policies on query features to multi-step cascades where a cheap model answers first and escalates only if its answer is judged unsatisfactory. Such cascades reduce inference cost by 50–98% while matching the best single model's accuracy [9]. This leverages both scaling-law and alignment research: larger models generally perform better but at higher per-token cost [6], while smaller, well-aligned models can outperform larger unaligned ones on preference metrics [28]. In Maestro, cascades can be realized at the LLM gateway, routing dynamically among on-premise open-weight models, mid-tier cloud APIs, and frontier proprietary models per a policy informed by query characteristics, historical performance, and business priorities [10, 38].

2.6.3 Prompt Compression

Prompt compression addresses the number of tokens processed per inference. Transformer attention scales quadratically with sequence length, so longer prompts cost more and are

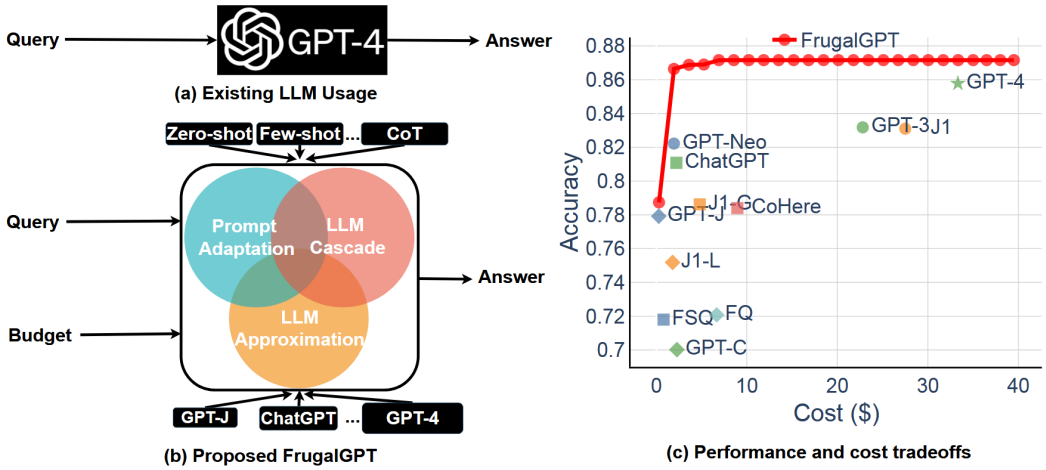

Figure 2.8: Overview of cost-reduction strategies in FrugalGPT, including prompt adaptation, LLM approximation, and LLM cascade. Adapted from Chen et al. [9].

slower [39]. In enterprise workflows, prompt length is often dominated not by the user query but by system instructions, conversation history, and retrieved RAG context [14, 10]. A compression step inserted between retrieval and generation summarizes or filters retrieved passages before the LLM call, reducing token usage without sacrificing answer quality.

The most academically grounded approach is the LLMLingua family. Jiang et al. [17] use a small surrogate model to score each token's saliency via conditional perplexity: tokens highly predictable from context carry little unique information and can be removed. Its pipeline comprises a budget controller setting the global compression ratio, iterative token-level pruning, and an alignment stage recovering any quality loss, achieving up to 20× compression while preserving factual content [17]. Pan et al. [29] reformulate compression as token classification using a bidirectional encoder trained on GPT-4 distillation data, making retention decisions from both directions - task-agnostic, faster, and more faithful than the perplexity-based approach [29]. Both exploit the same gap: a large fraction of tokens in any well-formed prompt are consumed by grammatical predictability rather than factual content.

2.6.4 Prompt Caching

Provider-side prompt caching reuses key-value (KV) cache entries for shared prompt prefixes - long system instructions, policy text, or tool definitions - so that repeated prefixes are billed at a reduced rate rather than recomputed in full. Major proprietary providers now support this mechanism, discounting cached input tokens substantially. Prompt caching is architecturally complementary to semantic caching but operates at the infrastructure layer: it requires exact prefix identity rather than semantic similarity, and reduces inference cost for repeated prefixes without bypassing the model call itself.

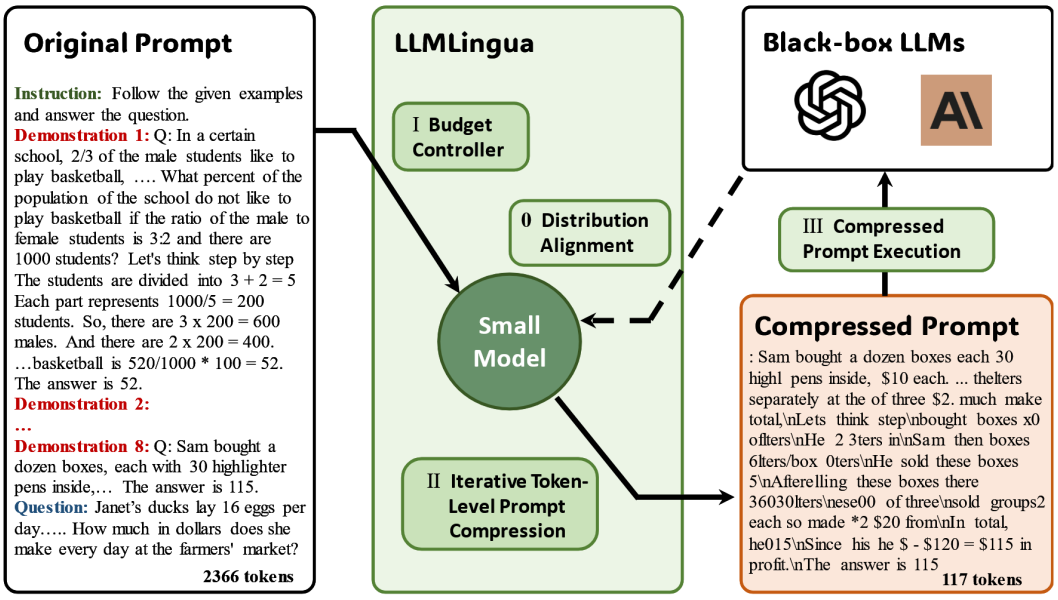

Figure 2.9: The LLMLingua coarse-to-fine prompt compression framework, comprising a budget controller, iterative token-level compression, and a distribution alignment stage. The small surrogate model scores token saliency via conditional perplexity; low-saliency tokens are iteratively removed before the compressed prompt is forwarded to the target LLM. Adapted from Jiang et al. [17].

2.6.5 Market Context and LLMFlation

Sardana and Frankle [3] introduce "LLMflation" to describe the rapid decline in inference costs: token prices have dropped by orders of magnitude for comparable capability as infrastructure, hardware, and model efficiency improved. However, lower unit prices do not automatically yield lower total spend. As LLMs become cheaper and more capable, organizations expand usage, apply models to more workflows, and supply more context per query - particularly in RAG and agentic settings [14, 10]. Demand growth and prompt-length inflation can outpace price cuts, mirroring cloud computing dynamics where falling resource prices often coincided with rising total expenditure [4]. Cost optimization must therefore be embedded in system architecture rather than treated as a transient concern, as semantic caching, cascades, prompt compression, and prompt caching remain valuable even as prices fall by targeting structural cost drivers: repeated queries, over-provisioned capacity, unnecessary tokens, and inefficient retrieval [5, 9, 14, 3].

2.6.6 Tokenomics in Practice: Representation and Style as Cost Levers

Beyond model- and retrieval-layer optimisations, token consumption is shaped by how structured data is serialised into prompts and how verbose the requested output is. TOON (Token-Oriented Object Notation) is a compact drop-in replacement for JSON in LLM prompts that declares column headers once and encodes rows as tab-delimited values,

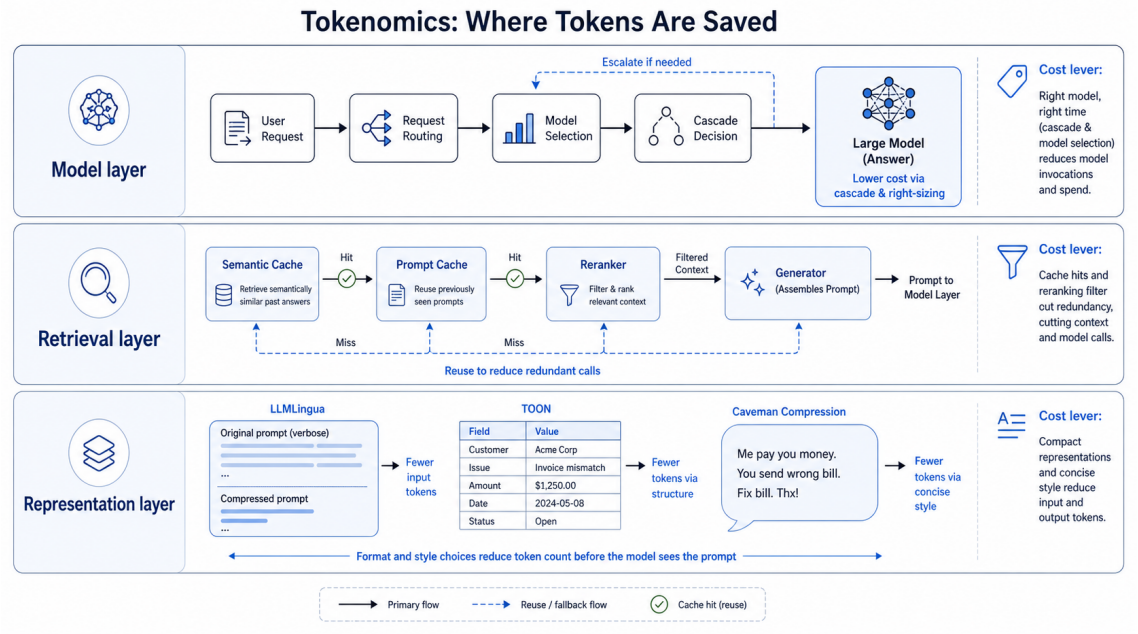

Figure 2.10: Three-layer tokenomics model for cost optimization in enterprise GenAI systems. The model layer controls cascade routing and dynamic model selection; the retrieval layer governs semantic caching, prompt caching, and reranking; the representation layer encompasses software compression (LLMLingua), format optimisation (TOON), and output style control (Caveman). Each layer targets a structurally distinct source of token consumption.

eliminating per-row key repetition [37], with practitioner benchmarks reporting 30–60% token reductions on flat tabular structures. Caveman Compression [7] targets the output side: LLMs can produce factually equivalent responses in a telegraphic register omitting articles, auxiliary verbs, and hedging, achieving roughly 75% output token reduction on structured tasks [7]; a subsequent distillation [15] reduced the meta-instruction from 552 to 85 tokens while preserving comparable savings. Both exploit the same gap as LLMLingua: tokens consumed by predictable surface forms rather than factual content. These findings originate in practitioner benchmarks rather than peer-reviewed studies, and are included as empirical evidence motivating formal investigation.

The three-layer tokenomics model can now be stated explicitly. At the *model layer*, cascade routing and dynamic model selection [9] invoke expensive frontier models only when warranted. At the *retrieval layer*, semantic caching [5], prompt caching, and reranking prevent redundant calls and filter irrelevant context. At the *representation layer*, compression such as LLMLingua [17], format optimisations such as TOON [37], and output style control such as Caveman Compression [7] eliminate tokens whose marginal semantic contribution is low. This taxonomy frames the engineering contributions of this dissertation, in which Maestro improvements address cost levers at all three layers within a unified, provider-portable architecture.

2.7 GenAI Operations and Governance

Deploying GenAI systems into production introduces operational challenges beyond traditional software engineering. Operational governance is not an optional layer atop a working system but a structural requirement whose absence compounds into systemic risk.

2.7.1 The Technical Debt Perspective on ML Operations

Sculley et al. [33] catalogued the *hidden technical debt* that accumulates in machine learning (ML) pipelines. ML code typically constitutes a small fraction of the system; the dominant cost lies in surrounding infrastructure - data ingestion, feature engineering, serving, monitoring, and configuration management - subject to *boundary erosion*, whereby coupling between data distributions, model behaviours, and downstream consumers degrades silently over time.

Particularly relevant to RAG is the CACE principle - "Change Anything, Changes Everything" - describing how altering a single upstream artefact, such as an embedding model or chunking strategy, may propagate unexpected behaviour through the system [33]. In a production RAG framework such as Maestro, which integrates a continuously evolving knowledge base, a swappable retrieval layer, and a provider-agnostic LLM interface, this entanglement is amplified. Without systematic instrumentation and operational discipline, technical-debt accumulation in GenAI systems is not merely possible but inevitable.

2.7.2 Evaluating RAG Pipeline Quality in Production

Classical metrics such as BLEU and ROUGE are inadequate for retrieval-augmented outputs, evaluating surface-level lexical overlap rather than factual faithfulness or contextual relevance [12]. Es et al. [12] introduced RAGAS (*Retrieval Augmented Generation Assessment*), a reference-free framework assessing RAG pipelines along four dimensions: *faithfulness* (grounding in retrieved context), *answer relevance* (addressing the question), *context precision* (retrieval focus), and *context recall* (retrieval coverage). Computed via an LLM-as-judge paradigm, these metrics enable automated, scalable evaluation embeddable directly into continuous integration and monitoring pipelines. RAGAS is the methodological basis for the output-quality objective in this thesis and the instrumentation through which Maestro's retrieval and generation quality is tracked in production.

2.7.3 RAGOps: Operationalising the Dual Lifecycle

Xu et al. [42] introduce retrieval-augmented generation operations (RAGOps) to characterise the discipline required to manage RAG pipelines in production. Unlike conventional deployments where the model artefact is the primary unit of management, RAG systems have a dual lifecycle: an *LLM lifecycle* governing model versioning, prompt management,

and inference configuration; and a *data lifecycle* governing the evolution, quality, and freshness of the external knowledge base. Documents are ingested, revised, or deprecated; embedding distributions shift as indexing evolves; and retrieval relevance may degrade independently of any model change [42]. Applied to Maestro, this motivates automated mechanisms for knowledge-base staleness detection, embedding index versioning, and retrieval-quality monitoring - capabilities not routinely addressed by LLMOps tooling built for static deployments. Xu et al. also report that roughly 60% of enterprise LLM deployments incorporate retrieval augmentation, underscoring the urgency of the discipline.

2.7.4 Financial Governance: The FinOps Lifecycle for AI

Token-priced inference is more dynamic and usage-dependent than the resource-reservation models of classical cloud workloads. Without deliberate financial governance, LLM systems are susceptible to overruns from prompt inefficiency, suboptimal routing, or unconstrained retrieval. The FinOps Foundation has extended its cloud financial management framework to these dynamics, defining a three-phase cycle: *Inform* (instrumenting token usage, call frequency, and cost per query), *Optimize* (implementing improvements such as model downsizing, semantic caching, and prompt compression), and *Operate* (embedding these into ongoing practice with cross-functional accountability) [13]. This cycle is continuous by design, positioning cost governance as a persistent discipline integrated into deployment and monitoring - not a one-time exercise.

This informs the cost-efficiency objective in this thesis and provides the structure within which the earlier cost-reduction techniques are situated as components of a managed, accountable process rather than isolated interventions. Together, the technical debt perspective, production evaluation methodology, RAGOps lifecycle, and FinOps governance cycle define the operational and financial framework within which the Maestro extensions proposed in this dissertation are situated.

3

System Description and Methodology

This chapter describes the technical platform on which the research contributions are built and the methodology governing their design, implementation, and evaluation. Section 3.1 characterises Maestro - the Closer GenAI Framework - in its pre-intervention state and identifies the structural gaps that motivate each objective. Section 3.2 formalises the research approach and the evaluation instruments applied to each objective.

3.1 System Description

3.1.1 Overview and Architectural Pattern

Maestro is a multi-repository, microservices-based RAG platform for deploying enterprise GenAI applications [10]. Although the LLM gateway is designed to abstract multiple model providers, all production deployments evaluated in this work operate on Microsoft Azure.

Independent FastAPI services communicate over HTTP with API-key authentication in a REST/JSON service mesh, with no message queue or event bus mediating inter-service calls. Service URLs are resolved through Azure App Configuration, secrets through Azure Key Vault, and all Python services share a base Docker image distributed via a shared Azure Container Registry. Infrastructure is provisioned declaratively with OpenTofu, and continuous integration and delivery are handled through Azure DevOps Pipelines.

The framework is implemented in Python 3.12 using FastAPI, LangChain, and Streamlit. As discussed in Section 2.4.1, LangChain provides Maestro's current orchestration primitives, a design decision that directly informs O4. The platform decomposes into four core runtime microservices, three user-facing surfaces, and two platform-layer repositories, summarised in Table 3.1.

This research focuses exclusively on the four core runtime microservices; the user-experience surfaces and the optional guardrails service are not modified and are described only as needed to characterise the system.

Table 3.1: Maestro service tiers and constituent repositories.

Tier	Repositories
Core runtime	`chatbot-backend`
	`llm-microservice`
	`chatbot-vector-db`
	`datastore-microservice`
Optional runtime	`guardrails-microservice`
User experiences	`chatbot-frontend`
	`chatbot-dashboard`
	`backoffice-angular`
Platform	`chatbot-opentofu`
	`demos-framework-root-container`

3.1.2 Core Services

Chatbot Backend (`chatbot-backend`). The backend is the sole orchestrator of the RAG pipeline and the single contact point for all frontend surfaces. It implements the full golden-path sequence - conversation-history retrieval, hybrid vector search, per-chunk relevance filtering, prompt assembly, and LLM invocation - through hand-crafted, non-composable orchestrators, each a standalone script with no shared primitive for branching, parallelism, or execution tracing. As established in Sections 2.3.2 and 2.4.1, this is characteristic of Naive-to-Advanced RAG pipelines that predate composable agent-graph runtimes; O4 addresses it by refactoring the orchestrators as reusable LangGraph graphs [8]. The service also performs no cost or token-consumption instrumentation, the structural gap addressed by O2.

LLM Microservice (`llm-microservice`). This service is a provider-agnostic LLM façade, routing generation requests to Azure OpenAI, OpenAI, Azure AI Services, and Google Gemini behind a stable internal API contract. A routing registry (`REQUEST_PER_SERVICE`) decouples model selection from orchestration logic, enabling model substitution without backend changes. Because queries are dispatched to models of varying capability and cost, the dynamic cascade routing and cost-aware model selection evaluated in O3 are architecturally feasible without redesigning the service [9]. O2 extends this service with Langfuse instrumentation to emit per-invocation token counts and monetary cost for every provider path.

Vector Database Service (`chatbot-vector-db`). This service owns document ingestion and semantic retrieval. It loads documents from Azure Blob Storage, splits them with LangChain's `RecursiveCharacterTextSplitter`, generates dense embeddings via `text-embedding-ada-002`, and stores the vectors in a MongoDB Atlas Vector Search index. At query time it supports both pure k-nearest-neighbour vector search and a hybrid mode combining vector similarity with a BM25-like keyword index, reflecting the Advanced

RAG mechanisms surveyed in Section 2.3.2 [14]. The semantic caching layer evaluated in O3 will be integrated at this retrieval stage, intercepting repeated semantically equivalent queries before the embedding and LLM invocation steps, consistent with the GPTCache architecture of Section 2.6.1 [5].

Datastore Microservice (`datastore-microservice`). This service provides a pluggable transactional data layer for chat history, session metadata, document records, and suggested questions, toggling between Azure Cosmos DB and MongoDB Atlas via a single `DATABASE` environment variable behind a shared API surface. It is not targeted by any of the four objectives but is an operational dependency of the backend orchestrator throughout the evaluation experiments.

3.1.3 Golden-Path Data Flow

The canonical interaction sequence, summarised in Table 3.2, defines the measurement surface for O2 and the optimisation targets for O3.

Table 3.2: Maestro golden-path interaction sequence.

Step	Action
1	The authenticated user submits a query via the Streamlit frontend.
2	The frontend issues `POST /v1/backend/ai-agent-response` to the backend with session, query, and selected provider.
3	The backend retrieves recent conversation history from the datastore.
4	The backend queries the vector database for the top-k similar chunks via hybrid retrieval.
5	The backend evaluates the relevance of each retrieved chunk via the LLM microservice; only passages judged relevant proceed.
6	The backend assembles the prompt and issues a generation call via the selected provider.
7	The answer and source citations are persisted and streamed back to the frontend.

No token counts, latency measurements, or cost signals are recorded at any stage of the current implementation - the structural gap directly motivating O2.

3.1.4 Connectivity and Protocol Gaps

Every external data source or capability in Maestro is currently integrated through a bespoke REST connector, with no shared discovery or invocation contract. The framework has no client layer through which it could consume external MCP-compliant servers without provider-specific adapter code. As discussed in Section 2.4.2, the MCP defines a universal JSON-RPC interface through which LLM hosts discover and invoke external tools and resources, replacing repeated point-to-point integration with a standardised,

plug-and-play connectivity model [2]. O1 addresses this gap by introducing an MCP integration layer that lets Maestro act as a gateway to external MCP servers.

3.2 Methodology

3.2.1 Research Approach

This work adopts a *Design Science Research* (DSR) methodology [16], the standard paradigm for research whose primary output is an engineered artifact evaluated in a real organisational context. DSR prescribes an iterative cycle of problem identification, artifact design, construction, and rigorous evaluation conducted in close collaboration with the environment that defines the problem. This is appropriate here because the contributions - an MCP integration layer, an observability pipeline, cost-reduction mechanisms, and a composable orchestration runtime - can only be assessed meaningfully under production conditions. The production Maestro deployment on Microsoft Azure serves simultaneously as engineering substrate, experimental environment, and source of evaluation data.

3.2.2 Dependency Ordering and Research Phases

The four objectives are addressed in an order that respects their dependency structure. O1 (MCP integration) is architecturally independent and can proceed in parallel with the instrumentation work of O2. O2 (cost telemetry pipeline) is a *hard prerequisite* for O3: without a quantitative production baseline of token cost, latency, and output quality there is neither a principled basis for selecting cost-reduction techniques nor a reference against which their effect can be measured. O4 (composable agent graphs) reuses the Langfuse instrumentation established in O2 for its trace-coverage evaluation, a deliberate architectural decision that avoids maintaining parallel instrumentation pipelines, as described in Section 2.7 [21].

Each objective follows three phases: *design*, in which the target architecture and tooling are specified; *implementation*, in which the artifact is built and integrated into the Maestro production stack; and *evaluation*, in which the artifact is assessed against the success criteria of Section 1.4 using the measurement instruments described below.

3.2.3 Implementation Approach per Objective

O1 - MCP Client Integration. An MCP client lets Maestro consume external MCP-compliant servers - such as MongoDB Atlas, Microsoft Learn, and Microsoft Fabric MCP - without bespoke adapter code, relying solely on the Anthropic MCP SDK and FastMCP for protocol-level discovery and invocation [2].

O2 - Unified Cost Telemetry Pipeline. Every LLM invocation path identified in a static audit of the Maestro codebase is instrumented using Langfuse as the primary LLM observability layer [21]. Each instrumented invocation exports cost per request, a full execution

trace, and - in sampling mode - RAGAS quality scores covering faithfulness and answer relevance [12]. Infrastructure-level signals are aggregated through the existing OpenTelemetry pipeline into Azure Monitor, providing a unified view across LLM and compute dimensions. The resulting cost baseline is validated against the corresponding Azure billing invoice before any O3 intervention begins, establishing its credibility as an evaluation reference.

O3 - Cost Reduction via Tokenomics Optimisation. The techniques implemented under O3 are selected from the production usage patterns surfaced by the O2 baseline, following the three-layer tokenomics framework presented in Section 2.6.6. A workload exhibiting high query repetition motivates semantic caching, implemented via MongoDB Atlas Semantic Cache or Azure Managed Redis [5]. A workload dominated by simple queries motivates cascade routing to smaller models via LangChain model routing [9]. Prompt compression via LLMLingua [17] is evaluated as a complementary technique targeting the representation layer. At least two techniques are implemented and empirically compared.

O4 - Composable Agent Graph Orchestration. Maestro's purpose-built orchestrators are refactored as reusable LangGraph graphs, sharing the Langfuse observability stack established in O2 [8]. At least two production workflows with structurally distinct topologies are refactored - one sequential and one with conditional branching or parallelism - demonstrating that the LangGraph pattern is expressive enough for diverse real-world cases. The refactored graphs are accompanied by unit tests, replacing the untested hand-crafted implementations.

3.2.4 Evaluation Strategy

O1. The MCP integration is validated through automated integration tests that verify successful invocation of external MCP server tools from Maestro, with no provider-specific adapter code in the invocation path, and is supported by a proof-of-concept deployment for each integrated external MCP server.

O2. The telemetry pipeline is evaluated on whether the identified LLM invocation paths are instrumented, each invocation exports cost, trace, and sampled RAGAS scores, and the Langfuse-reported costs reconcile with the Azure billing invoice before any O3 intervention begins.

O3. Cost-reduction effectiveness is measured as the percentage reduction in cost-per-query relative to the O2 baseline over a set of production queries partitioned into a control group and an experimental group. Quality preservation is evaluated via the Two One-Sided Tests (TOST) procedure [19] on RAGAS faithfulness and answer relevance scores. TOST is adopted in preference to a standard two-sample t-test because the research question is one of *equivalence* - demonstrating that the experimental and control quality distributions are

sufficiently close to be indistinguishable - whereas a non-significant t-test only fails to detect a difference and cannot, on its own, support a positive claim of quality preservation [20].

O4. The orchestration artifacts are evaluated on refactoring at least two workflows with structurally distinct topologies, achieving substantial Langfuse trace coverage of execution steps, reducing lines of code relative to the original hand-crafted implementations, and adding unit tests where the originals had none.

3.2.5 Experimental Controls and Validity Threats

The primary threat to *internal validity* in O3 is the non-stationarity of the production query distribution: if user behaviour shifts between the baseline and intervention period, observed cost differences may reflect workload change rather than technique efficacy. This is mitigated by partitioning queries into control and experimental groups within the same observation window, stratified to preserve query-type distribution.

The primary threat to *construct validity* is the use of Langfuse-reported token counts as a proxy for actual cost; this is addressed by the O2 validation requiring reconciliation with Azure billing data before the instrument serves as an evaluation reference.

External validity is bounded by the single-environment nature of the evaluation: all implementation and measurement work is conducted on the Azure deployment of Maestro. Architectural patterns and cost-reduction findings are expected to transfer to equivalent RAG deployments on other cloud platforms, but cross-environment equivalence is not empirically validated within the scope of the present work.

4

Work Plan

This chapter presents the work plan governing the remaining research activities. Section 4.1 describes the three active phases through which the four objectives are pursued. Section 4.2 situates those phases within a concrete timeline. Section 4.3 specifies the deliverables and success criteria associated with each objective.

4.1 Research Phases

The work is organised across three active phases, executed from May 2026 onward. Phase sequencing reflects the hard dependency order established in Section 3.2.2: O1 must precede O2, O2 is a strict prerequisite for O3, and O4 is conditional on the substantial completion of O3.

Phase 1 - Technical Development. This phase encompasses the implementation of O1 (MCP Client Integration) and O2 (Unified Cost Telemetry Pipeline). O1 is architecturally self-contained and proceeds first; its completion establishes a standardised connectivity layer for the Maestro framework without dependency on any other objective. O2 follows and constitutes the most critical milestone of the entire work plan: it instruments 100% of LLM invocation paths with Langfuse and validates reported costs against the Azure billing invoice, producing the quantitative production baseline without which no cost-reduction work can be selected, executed, or evaluated. No activity pertaining to O3 or O4 commences before the O2 baseline is available.

Phase 2 - Results Consolidation and Technical Writing. This phase covers the implementation of O3 (Tokenomics Cost Reduction), its rigorous statistical evaluation, and, contingent on time availability, O4 (LangGraph Composable Agent Orchestration). The specific cost-reduction techniques applied under O3 - drawn from semantic caching, LLMLingua prompt compression, and cascade routing - are selected on the basis of the production usage patterns surfaced by the O2 baseline, following the three-layer tokenomics framework introduced in Section 2.6.6. Statistical equivalence of output quality is assessed via Two One-Sided Tests (TOST) [19] on RAGAS faithfulness and answer relevance. O4 is pursued

only if O3 is substantially complete before the end of November 2026; it represents an independent architectural improvement rather than a dependency of O3. Chapter writing runs in parallel with implementation throughout this phase to prevent bottlenecks in the final period.

Phase 3 - Final. This phase is devoted to the consolidation of all written chapters into a coherent manuscript, iterative revision with both supervisors, formal approval, public defence, and official submission to NOVA NOVA School of Science and Technology (FCT). No new implementation work is initiated during this phase.

4.2 Timeline

The timeline spans from May 2026 to March 2027 and is summarised in Figure 4.1. The schedule reflects both the dependency ordering described in Section 4.1 and a deliberate strategy of overlapping implementation with writing wherever safe to do so.

O1, already under way from the beginning of May 2026, runs through the end of July. O2 starts at the beginning of July, overlapping with the final weeks of O1 - a safe overlap given the architectural independence of the two objectives - and continues through the end of August. This overlap is intentional: it prevents an idle gap in development activity while O1 is being finalised, without compromising the O2 instrumentation work that must be complete before O3 begins.

From September 2026 onward, the work plan bifurcates. O3 implementation begins at the start of September, aligned with the confirmed availability of the O2 baseline, and runs through the middle of November. The TOST statistical evaluation of O3 results occupies the second half of October through the end of November, overlapping the final weeks of O3 implementation to allow statistical analysis to proceed as experimental results accumulate. Chapter 3 (System and Methodology) is written in parallel from the beginning of September to the middle of October, followed by Chapter 4 (Results), which runs from the beginning of November through the end of December. O4 is scheduled from the beginning of November to the middle of December, subject to the satisfactory completion of O3.

Phase 3 begins at the start of January 2027, with the remainder of that month and February allocated to full manuscript revision, supervisor approval, public defence, and official submission to NOVA FCT, targeting completion at the beginning of March 2027.

4.3 Deliverables

Each research objective is associated with a concrete, measurable deliverable and a set of success criteria that operationalise the objective conditions stated in Section 1.4. Table 4.1 consolidates this information.

Figure 4.1: Gantt chart of the research work plan (May 2026 - March 2027).

Table 4.1: Research objectives, deliverables, and success criteria.

Objective	**Deliverable**	**Success criteria**
O1 - MCP client integration	MCP client layer for Maestro	Maestro consumes external MCP servers without provider-specific adapter code
O2 - Unified cost telemetry pipeline	Langfuse and Azure Monitor instrumentation	The LLM invocation paths are instrumented with per-invocation cost and execution trace, the reported costs reconcile with the Azure billing invoice, and the baseline is available before O3 begins
O3 - Cost reduction via tokenomics optimisation	Benchmark of complementary tokenomics techniques on the production workload	Cost-per-query is reduced relative to the O2 baseline while quality equivalence on RAGAS faithfulness and answer relevance is demonstrated via Two One-Sided Tests (TOST)
O4 - Composable agent graph orchestration *(if time permits)*	LangGraph refactors of production workflows with unit tests	Production workflows with structurally distinct topologies are refactored as LangGraph graphs, with substantial Langfuse trace coverage and reduced lines of code relative to the hand-crafted implementations

The dependency structure embedded in this table is deliberate. O2 is listed before O3 because the success criterion of O3 - a reduction measured relative to the O2 baseline - is undefined until O2 is complete and validated. O4 carries an explicit contingency flag, consistent with the priority ordering stated in Section 1.4: it is pursued only after O3 is substantially complete, and its omission does not affect the validity of the dissertation's primary contributions.

Bibliography

[1] Anonymous. "Mitigating Bias and Hallucinations in LLMs through Prompt Engineering and Knowledge-Grounded Approaches in Healthcare Domain: A Systematic Literature Review". In: *IEEE Access* (2025). States explicitly: "MCP emerges as a promising but underexplored tool for dynamic knowledge integration". DOI: 10.1109/ICATC68823.2025.11407841 (cit. on p. 3).

[2] Anthropic. *Model Context Protocol: Specification*. Tech. rep. Anthropic, 2024. URL: https://www.anthropic.com/news/model-context-protocol (cit. on pp. 1, 3, 5, 7, 13, 27).

[3] G. Appenzeller. *Welcome to LLMflation: LLM inference cost is going down fast*. Andreessen Horowitz; accessed May 2026. 2024-11. URL: https://a16z.com/llmflation-llm-inference-cost/ (cit. on pp. 4, 18, 20).

[4] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. H. Katz, A. Konwinski, G. Lee, D. A. Patterson, A. Rabkin, I. Stoica, and M. Zaharia. "A View of Cloud Computing". In: *Communications of the ACM* 53.4 (2010-04), pp. 50–58. DOI: 10.1145/1721654.1721672 (cit. on pp. 16, 17, 20).

[5] F. Bang et al. "GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings". In: *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software* (2023), pp. 212–218. URL: https://aclanthology.org/2023.nlposs-1.24/ (cit. on pp. 4, 6, 18, 20, 21, 26, 28).

[6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020-12, pp. 1877–1901. DOI: 10.48550/arXiv.2005.14165 (cit. on pp. 1, 9, 10, 18).

[7] J. Brussee. *Caveman: A Semantic Compression Skill for LLMs*. Claude Code skill; widely replicated; accessed May 2026. 2025. URL: https://github.com/wilpel/caveman-compression (cit. on p. 21).

[8] H. Chase. *LangChain: Building Applications with Large Language Models*. https://langchain.com. Framework documentation and whitepaper. Accessed: 2026. 2022 (cit. on pp. 6, 13, 25, 28).

[9] L. Chen, M. Zaharia, and J. Zou. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*. 2023. arXiv: 2305.05176 [cs.CL] (cit. on pp. 4, 6, 18–21, 25, 28).

[10] Closer Consulting. *Maestro GenAI Framework*. Tech. rep. Internal technical documentation and client deployment case studies. Confidential practitioner source. Closer Consulting, 2025. URL: %5Curl%7Bhttps://closerconsultoria.sharepoint.com/:p:/r/sites/GenAIFramework-DEMO/_layouts/15/Doc.aspx?sourcedoc=%7B42094324-FC65-4A0D-9734-58B50056E83C%7D&file=Closer_Maestro_Framework_Closer_Version.pptx&action=edit&mobileredirect=true%7D (cit. on pp. 2–4, 15–20, 24).

[11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*. 2019-06, pp. 4171–4186. DOI: 10.18653/v1/N19-1423 (cit. on p. 9).

[12] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert. "RAGAS: Automated Evaluation of Retrieval Augmented Generation". In: *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL)*. 2024, pp. 150–158. DOI: 10.18653/v1/2024.eacl-demo.16 (cit. on pp. 5, 7, 22, 28).

[13] FinOps Foundation. *FinOps for AI Overview*. Tech. rep. Working group publication. Accessed: May 2026. FinOps Foundation, 2024. URL: https://www.finops.org/wg/finops-for-ai-overview/ (cit. on p. 23).

[14] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2023-12. DOI: 10.48550/arXiv.2312.10997 (cit. on pp. 4, 11, 12, 19, 20, 26).

[15] K. Guzik. *Caveman-Micro: Distilled Brevity Instruction for LLMs*. Benchmark conducted on Claude Sonnet and Opus; accessed May 2026. 2026. URL: https://github.com/kuba-guzik/caveman-micro (cit. on p. 21).

[16] A. R. Hevner, S. T. March, J. Park, and S. Ram. "Design Science in Information Systems Research". In: *MIS Quarterly* 28.1 (2004), pp. 75–105. DOI: 10.2307/25148625 (cit. on p. 27).

[17] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Yu. "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2023. URL: https://arxiv.org/abs/2310.05736 (cit. on pp. 4, 6, 19–21, 28).

[18] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih. "Dense Passage Retrieval for Open-Domain Question Answering". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020-11, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550 (cit. on p. 11).

[19] D. Lakens. "Equivalence Tests: A Practical Primer for t Tests, Correlations, and Meta-Analyses". In: *Social Psychological and Personality Science* 8.4 (2017), pp. 355–362. DOI: 10.1177/1948550617697177 (cit. on pp. 6, 28, 30).

[20] D. Lakens, A. M. Scheel, and P. M. Isager. "Equivalence Testing for Psychological Research: A Tutorial". In: *Advances in Methods and Practices in Psychological Science* 1.2 (2018), pp. 259–269. DOI: 10.1177/2515245918770963 (cit. on pp. 6, 29).

[21] Langfuse. *Langfuse: Open Source LLM Engineering Platform*. Accessed: June 2026. 2024. URL: https://langfuse.com (cit. on p. 27).

[22] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer. "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020-07, pp. 7871–7880. DOI: 10.18653/v1/2020.acl-main.703 (cit. on p. 11).

[23] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020-12, pp. 9459–9474. DOI: 10.48550/arXiv.2005.11401 (cit. on p. 11).

[24] J. Liu. *LlamaIndex*. 2022. URL: https://github.com/jerryjliu/llama_index (cit. on p. 13).

[25] J. M. Lourenço. *The `aidisclose` package: Generative AI disclosure checklist and statements*. Version 1.6.2. 2025. URL: https://ctan.org/pkg/aidisclose (cit. on p. iii).

[26] T. Mikolov, K. Chen, G. Corrado, and J. Dean. *Efficient Estimation of Word Representations in Vector Space*. 2013-09. DOI: 10.48550/arXiv.1301.3781 (cit. on p. 8).

[27] Model Context Protocol Contributors. *Model Context Protocol — Reference Documentation*. https://modelcontextprotocol.io. Canonical technical specification. Accessed: 2026. 2024 (cit. on pp. 3, 5, 7, 13, 14).

[28] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 35. 2022-12, pp. 27730–27744. DOI: 10.48550/arXiv.2203.02155 (cit. on pp. 10, 11, 18).

[29] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang. "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression". In: *arXiv preprint arXiv:2403.12968* (2024). URL: https://arxiv.org/abs/2403.12968 (cit. on pp. 4, 19).

[30] D. Petcu. "Portability and Interoperability between Clouds: Challenges and Case Study". In: *Towards a Service-Based Internet*. Lecture Notes in Computer Science 8135 (2013), pp. 62–74. DOI: 10.1007/978-3-642-45015-0_6 (cit. on pp. 16, 17).

[31] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams. "A Systematic Mapping Study of Infrastructure as Code Research". In: *Information and Software Technology* 108 (2019-04), pp. 65–77. DOI: 10.1016/j.infsof.2018.12.004 (cit. on p. 17).

[32] N. Reimers and I. Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2019-11, pp. 3982–3992. DOI: 10.18653/v1/D19-1410 (cit. on p. 12).

[33] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison. "Hidden Technical Debt in Machine Learning Systems". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 28. 2015. URL: https://papers.neurips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf (cit. on p. 22).

[34] Y. Suchikova, N. Tsybuliak, J. A. T. da Silva, and S. Nazarovets. "GAIDeT (Generative AI Delegation Taxonomy): A Taxonomy for Humans to Delegate Tasks to Generative Artificial Intelligence in Scientific Research and Publishing". In: *Accountability in Research* (2025), pp. 1–27. DOI: 10.1080/08989621.2025.2544331 (cit. on p. iii).

[35] The LaTeX Project team. *LaTeX - A document preparation system*. 1985. URL: https://www.latex-project.org (visited on 2025-12-29) (cit. on p. iii).

[36] Thoughtworks. *Technology Radar Volume 33*. Technical Report. Places MCP in "Trial"; identifies rapid industrial adoption of MCP-powered agents as a defining theme of 2025. Thoughtworks, 2025-11. URL: https://www.thoughtworks.com/radar (cit. on p. 3).

[37] *TOON: Token-Oriented Object Notation*. Practitioner serialization format for LLM prompt token efficiency; accessed May 2026. 2025. URL: https://openapi.com/blog/what-the-toon-format-is-token-oriented-object-notation (cit. on p. 21).

[38] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. 2023-07. DOI: 10.48550/arXiv.2307.09288 (cit. on pp. 10, 11, 18).

[39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017-12. DOI: 10.48550/arXiv.1706.03762 (cit. on pp. 8, 9, 19).

[40] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen. *A Survey on Large Language Model Based Autonomous Agents*. 2023. DOI: 10.48550/arXiv.2308.11432 (cit. on pp. 2, 4, 5).

[41] C.-J. Wu, R. Raghavendra, U. Gupta, B. Acun, N. Ardalani, K. Maeng, G. Chang, F. A. Behram, J. Huang, C. Bai, M. Gschwind, A. Gupta, M. Ott, A. Melnikov, S. Candido, D. Brooks, G. Chauhan, B. Lee, H.-H. S. Lee, B. Agrawal, M. Fixler, S. Sheahan, S. Sridharan, A. M. Zain, and M. Smelyanskiy. "Sustainable AI: Environmental Implications, Challenges, and Opportunities". In: *Proceedings of Machine Learning and Systems (MLSys)*. Vol. 4. 2022, pp. 795–813. URL: https://arxiv.org/abs/2111.00364 (cit. on pp. 2, 3).

[42] X. Xu et al. *Operating and Managing Retrieval-Augmented Generation Pipelines*. 2025. arXiv: 2506.03401 [cs.SE] (cit. on pp. 22, 23).

[43] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*. 2023. URL: https://arxiv.org/abs/2210.03629 (cit. on pp. 13, 14).